S.No: 4	Exp. Name: ***Minimum Spanning Tree using Kruskal's Algorithm***	Date:2026-07-07

ID: 25070521290

Aim:

Write a C program to construct the Minimum Spanning Tree (MST) of a connected, weighted, undirected graph using Kruskal's Algorithm.

You are provided with a driver code. Complete the function **kruskalMST()** so that the program correctly finds and prints the Minimum Spanning Tree and its total cost. The graph is represented using an adjacency matrix.

Implement the function:

```
void kruskalMST(int **cost, int V)
```

Where,

- $V \rightarrow$ Number of vertices in the graph
- $\text{cost} \rightarrow$ Adjacency matrix storing edge weights

Input Format:

- The first line of input reads an integer V by prompting "No of vertices: ", representing the number of vertices in the graph.
- The next $V \times V$ integers represent the adjacency matrix of the graph with a heading "Adjacency matrix:"
- $\text{cost}[\text{i}][\text{j}]$ indicates the weight of the edge between vertex i and vertex j
- If there is no edge between two vertices, the weight should be given as 9999 (representing infinity).
- The graph is undirected, so the matrix is symmetric

Output Format:

1.The output should display each edge selected for the Minimum Spanning Tree in the order they are chosen in the below format:

```
Edge k:(u, v) cost:w
```

Where:

- k is the edge number starting from 0
- u and v are the vertices connected by the edge
- w is the weight of the edge

2.In the last line the program should print the total minimum cost of the MST as follows:

```
Minimum cost= X
```

Where X is the total cost of the Minimum Spanning Tree.

Constraints:

- $1 \leq V \leq 100$
- Edge weights are non-negative integers
- The adjacency matrix is dynamically allocated
- Exactly $V - 1$ edges must be included in the MST

Note:

- Process the edges in ascending order of their weights. For edges with the same weight, follow their row-wise order in the upper triangular part of the adjacency matrix.
- Refer to the visible test cases to strictly match the input and output layout.

Source Code:

minCostFinding.c

```
#include <stdio.h>
#include <stdlib.h>
#include <limits.h>


int find(int parent[], int i) {
    while (parent[i] != i)
        i = parent[i];
    return i;
}

void Union(int parent[], int x, int y) {
    parent[y] = x;
}

void kruskalMST(int **cost, int V) {
    int parent[100];
    int mincost = 0;

    for (int i = 0; i < V; i++)
        parent[i] = i;

    int edge = 0;

    while (edge < V - 1) {
        int min = INT_MAX;
        int a = -1, b = -1;

        // Find minimum edge
        for (int i = 0; i < V; i++) {
            for (int j = 0; j < V; j++) {
                if (cost[i][j] < min) {
                    min = cost[i][j];
                    a = i;
                    b = j;
                }
            }
        }

        int u = find(parent, a);
        int v = find(parent, b);

        if (u != v) {
            printf("Edge %d:(%d, %d) cost:%d\n", edge, a, b, min);
            mincost += min;
            Union(parent, u, v);
            edge++;
        }

        cost[a][b] = INT_MAX;
        cost[b][a] = INT_MAX;
    }
```


ID: 25070521290

```
    printf("Minimum cost= %d\n", mincost);
}



int main() {
    int V;
    printf("No of vertices: ");
    scanf("%d", &V);

    int **cost = (int **)malloc(V * sizeof(int *));
    for (int i = 0; i < V; i++)
        cost[i] = (int *)malloc(V * sizeof(int));

    printf("Adjacency matrix:\n");
    for (int i = 0; i < V; i++)
        for (int j = 0; j < V; j++)
            scanf("%d", &cost[i][j]);

    kruskalMST(cost, V);

    for (int i = 0; i < V; i++)
        free(cost[i]);
    free(cost);

    return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
No of vertices:
5
Adjacency matrix:
9999 2 9999 9999 5
2 9999 3 9999 9999
9999 3 9999 4 9999
9999 9999 4 9999 9999
5 9999 9999 9999 9999
Edge 0:(0, 1) cost:2
Edge 1:(1, 2) cost:3
Edge 2:(2, 3) cost:4
Edge 3:(0, 4) cost:5
Minimum cost= 14

Test Case - 2
User Output
No of vertices:
4

Adjacency matrix:
9999 3 6 3
3 9999 5 2
6 5 9999 4
3 2 4 9999
Edge 0:(1, 3) cost:2
Edge 1:(0, 1) cost:3
Edge 2:(2, 3) cost:4
Minimum cost= 9